The Peleka Universal Rate Model (PURM): A Technical Specification for the Pricing Engine

Macharia Barii

November 3, 2026

Abstract

This document specifies an implementable pricing engine for contract logistics in markets with imperfect endpoint addressability. It operationalises the theory developed in four companion papers—*The Addressability Threshold and the Drop Rate*, *Address Regimes and Moat Durability*, *The Drop Proof Rule*, and *The Universal Pricing Algorithm*—into a concrete algorithm with defined data structures, scoring routines, a rate card, a constrained optimisation, six acceptance gates, a trip-level marginal insertion procedure, a corridor viability check, and a long-haul variant. The engine is universal in structure and local in parameters: the same computational object prices a KSh 600 kiosk drop in Nairobi and a forty-tonne trunk haul from Mombasa, with only the inputs changing. The specification includes formal statements of the fixed-point problem, a proof of monotonicity on which the bisection solver depends, twelve acceptance tests, and an honest treatment of the cold-start problem. The engine's defining property is that it can return *"no stable price exists"* as a valid output—the discipline the previous generation of logistics companies lacked.

Contents

1 Purpose and Scope **1**

2 Relationship to the Theory **2**

3 Notation **2**

4 Architecture **2**

5 Data Model **3**
5.1 Entities . . . 3
5.2 Invariants . . . 4

6 Scoring Layer **4**
6.1 Addressability . . . 4
6.2 Success probability with shrinkage . . . 4
6.3 Service time . . . 5
6.4 Expected loss . . . 5

7 Rate Card **5**

8 The Floor Function **6**

9 The Density–Price Fixed Point **7**
9.1 The problem 7
9.2 Monotonicity 7
9.3 Existence and solution 7
9.4 Solver 8

10 The Six Gates **8**
10.1 The verdict 8
10.2 The four responses 9

11 Trip Level: Marginal Insertion **9**

12 Corridor Level **10**

13 Long Haul **10**

14 Complete Algorithm **11**

15 Acceptance Tests **11**

16 Calibration and Cold Start **12**

17 Complexity and Performance **12**

18 Limitations **12**

19 Conclusion **13**

1 Purpose and Scope

This document specifies a pricing engine that answers, for any logistics job presented to the firm, one of four questions:

1. **Prove:** is this job provable at a price the customer will accept?
2. **Price:** if so, at what price?
3. **Prepare:** if not, what change of state would make it provable?
4. **Pass:** or is no change sufficient, and the job must be declined?

The engine is a decision procedure, not a price oracle. It does not tell you what a customer will pay; it tells you whether there exists a price at which the job contributes, and what that price is if it exists.

Scope. The engine applies to any market satisfying the five scope conditions of the companion theory paper: imperfect static addressing, search cost that scales with asset value, a partially informal supplier base, slow state address provision, and non-degenerate recipient variance. Where those conditions fail, the engine's search and risk terms collapse and it degenerates to standard contribution-margin pricing, which is correct behaviour.

Out of scope. The engine does not model competitive response, strategic client value beyond the transaction, or the cost of capital. These enter through the ceiling and through management overrides, and are explicitly outside the algorithm.

2 Relationship to the Theory

The engine is the computational form of four results:

Theoretical result	Computational manifestation
The rate primitive $R(a)$	Scoring layer: addressability, posterior success, service-time statistics
The invariant (master inequality)	Gate 3, computed from the scoring layer
The bootstrap/moat distinction	The registry as a cost, the event stream as the audit log
The multi-level hierarchy	Drop $\to$ trip $\to$ corridor $\to$ long haul, each with its own test
The fixed-point problem	Section 9; the bisection solver
The long-haul multiplicative reliability	Section 13; joint probability gate
The four responses	Section 10; the verdict enum

Nothing in this specification is a free parameter of the theory. Every quantity is either derived (the floor), measured (addressability, success probability), or configured (the rate card, cost parameters). The engine is falsifiable in the same sense the theory is: if it produces systematically wrong verdicts, either the parameters are wrong or the theory is.

3 Notation

Symbol	Meaning
a	Addressability of a place, $a \in (0, 1]$
$p(a)$	First-attempt success probability, increasing in a
$A(a)$	Addressing surcharge $= 1 + \mu(1 - a)/a$, $\mu < \lambda$
λ	Search cost coefficient (opportunity cost of the asset per minute)
S	Fuel surcharge multiplier
T	Tier multiplier
B	Base rate from the rate card
n	Net service fee (pre-VAT)
P	Client price (VAT-inclusive), $P = (1 + v)n$
N	Drops per trip (density)
$f(N)$	Floor price at density N
n^*	Break-even drops per trip
Φ	Principal burden coefficient $= v(1 - \kappa) + r\Delta t$
C_p	Partner capacity cost per trip
c_0	Dispatch cost per trip
c_r	Reattempt cost per drop
c_{var}	Variable cost per drop
$\mathbb{E}[\text{loss}]$	Expected cargo loss per drop
v	VAT rate
κ	Share of VAT-registered partners

4 Architecture

The engine runs in three execution contexts with strictly separated responsibilities.

Context	Responsibility	Latency budget
Offline (nightly batch)	Recompute scores from the event stream; decay stale places; update posteriors	Unbounded
Online (per quote)	Score lookup, gates, verdict	≤ 50 ms
Batch (per planning cycle)	Route assignment, corridor viability, replanning	Minutes

Invariant. The online path only *reads* scores. It never computes them. This is what keeps quoting fast, makes decisions reproducible, and separates the data pipeline from the commercial decision.

5 Data Model

5.1 Entities

```
class Verdict(Enum):
    PROVE = "prove" # all gates pass; dispatch
    PRICE = "price" # interval non-empty; price at floor
    PREPARE = "prepare" # state change makes it provable
    PASS = "pass" # not provable at any acceptable price

@dataclass
class Place:
    place_id: str
    lat: float; lon: float
    landmark: str
    corridor: str
    zone: str # "A" | "B" | "C"
    a_geocode: float # 0..1, from field verification
    a_landmark: float # 0..1, from field verification
    a_gate: float # 0..1, from field verification
    a_reach: float # 0..1, from recipient history
    a_first_attempt: float # 0..1, from event stream
    n_deliveries: int
    last_verified_days: int
    alpha: float = 1.0 # Beta posterior on success
    beta: float = 1.0

@dataclass
class DeliveryPoint:
    dp_id: str
    account_id: str
    place_id: str
    ap_id: str # access point within the place
    recipient_phone: str
    alpha: float = 1.0 # dp-level posterior
    beta: float = 1.0

@dataclass
class Job:
    job_id: str
    account_id: str
    place_id: str
    dp_id: str
    weight_kg: float
    volume_m3: float
    distance_km: float
    tier: str # standard|priority|secured|highvalue
```

```
    declared_value: float # KSh, for E[loss]
    window_width_min: int # requested SLA window
    segment: str # for elasticity calibration
    is_long_haul: bool = False

@dataclass
class QuoteDecision:
    job_id: str
    verdict: Verdict
    net_price: float
    client_price: float
    floor: float
    ceiling: Optional[float]
    density: float
    gates: dict # gate -> (pass, value, threshold)
    reason: str
    scored: dict # all intermediates, for audit
```

5.2 Invariants

- A `Place` is shared across customers. Customer identity lives on `DeliveryPoint`.
- Every `QuoteDecision` is appended to an immutable log, including all intermediate terms. This log is auditable and is the same discipline as the event stream.
- Posteriors are stored on the record; all other statistics are computed from the event stream at scoring time.

6 Scoring Layer

Scoring runs offline, nightly. It reads the event stream and produces the derived quantities the online path consumes.

6.1 Addressability

```
A_WEIGHTS = dict(geocode=0.30, landmark=0.25, gate=0.15,
    reach=0.20, first_attempt=0.10)
DECAY_HALFLIFE_DAYS = 90

def addressability(pl: Place, today: int) -> float:
    """Static + dynamic composition, with freshness decay."""
    a = (A_WEIGHTS["geocode"] * pl.a_geocode
         + A_WEIGHTS["landmark"] * pl.a_landmark
         + A_WEIGHTS["gate"] * pl.a_gate
         + A_WEIGHTS["reach"] * pl.a_reach
         + A_WEIGHTS["first_attempt"] * pl.a_first_attempt)
    decay = 0.5 ** (pl.last_verified_days / DECAY_HALFLIFE_DAYS)
    return max(0.01, a * decay)
```

The weights implement the static/dynamic decomposition of the theory: 0.70 is static (verifiable in advance), 0.30 is dynamic (earned through delivery). The freshness decay is not optional—a registry without decay is a liability, because it produces confidently wrong scores.

6.2 Success probability with shrinkage

The success probability at a delivery point is a Beta posterior, shrunk toward the place prior when the delivery point has few observations.

```
def p_hat(pl: Place, dp: DeliveryPoint, k: float = 5.0) -> float:
    """
    Beta posterior at dp-level, shrunk toward place prior.
    k = shrinkage strength in equivalent prior observations.
    """
    place_mean = pl.alpha / (pl.alpha + pl.beta)
    dp_n = dp.alpha + dp.beta - 2.0
    dp_mean = (dp.alpha - 1.0) / dp_n if dp_n > 0 else place_mean
    return (dp_n * dp_mean + k * place_mean) / (dp_n + k)
```

This implements the theory's shrinkage result: a new delivery point at a well-verified place inherits the place prior and can be quoted a tight window on day one.

6.3 Service time

```
def tau_stats(pl: Place, ap_id: str, hour: int) -> tuple[float, float]:
    """Mean and variance of service time, from the event stream."""
    events = query_events(pl.place_id, ap_id, hour_bucket(hour))
    if len(events) < MIN_EVENTS_FOR_STATS:
        return DEFAULT_TAU_MEAN, DEFAULT_TAU_VAR
    return mean(events.dwell_min), var(events.dwell_min)
```

Service time is computed rather than stored, because the statistics at a place improve continuously and a stored value would be stale. The cold-start default is used only until the place has accumulated three observations.

6.4 Expected loss

```
RHO = dict(fraud=0.008, theft=0.006, damage=0.004) # per corridor

def expected_loss(job: Job, dp: DeliveryPoint, pl: Place) -> float:
    if job.declared_value < HIGH_VALUE_THRESHOLD:
        return 0.0
    trust = p_hat(pl, dp) # 0..1
    mult = 2.0 - trust # trust=1 -> 1.0x, trust=0 -> 2.0x
    rho = sum(RHO.values()) * mult
    controls = security_control_factor(job.tier)
    return rho * controls * job.declared_value
```

The `trust` multiplier is the recipient trust graph entering the model: a proven recipient halves the expected loss relative to an unknown one. The controls factor reflects the security tier—OTP, ID check, named driver—and is the only way to reduce expected loss below what recipient history provides.

7 Rate Card

The rate card sets the *form* of the price. The floor sets the *level*. Both are needed.

```
RATE_CARD = {
    "A": dict(r_km=30, F=400, M=600),
    "B": dict(r_km=22, F=350, M=500),
    "C": dict(r_km=18, F=300, M=450),
}
D0 = 5.0
K_BANDS = [(250, 3.0), (1000, 2.2), (float("inf"), 1.6)]
TIER = {"standard": 1.00, "priority": 1.30,
    "secured": 1.18, "highvalue": 1.50}
```

```
def chargeable_weight(job: Job) -> float:
    volumetric = job.volume_m3 * 167.0
    return max(job.weight_kg, volumetric)

def k_rate(w: float) -> float:
    for cap, rate in K_BANDS:
        if w <= cap: return rate
    return K_BANDS[-1][1]

def base_rate(job: Job, pl: Place) -> float:
    z = RATE_CARD[pl.zone]
    delta = max(0.0, job.distance_km - D0)
    w = chargeable_weight(job)
    b = z["F"] + z["r_km"] * delta + k_rate(w) * w
    return max(z["M"], b)

def fuel_mult(f_now: float, f_base: float, beta: float = 0.4) -> float:
    return 1.0 + beta * (f_now - f_base) / f_base

def addressing_mult(a: float, mu: float, lam: float) -> float:
    assert mu < lam, "pass-through cannot exceed search cost"
    return 1.0 + mu * (1.0 - a) / a
```

The final client price for an accepted job is:

$$P = (1 + v) \cdot S \cdot T \cdot A(a) \cdot \max\bigl(p^{*}, B\bigr) \tag{1}$$

where p^{*} is the density-solved floor from Section 9. The rate card is what the customer sees; the floor is what the business needs. When the floor exceeds the base, the customer is charged the floor and a warning is logged—the job is priced for what it costs, not for what the rate card says.

8 The Floor Function

Definition 1 (Floor price)**.** *Given cost parameters and a place, the floor price at density N is*

$$f(N) = \frac{c_{var} + \Phi C_p / N + (1 - \hat{p}) c_r + \mathbb{E}[loss]}{\hat{p} \cdot A(a)}. \tag{2}$$

The critical term is $\Phi C_p / N$: the principal burden is fixed per trip, so its per-drop share falls as density rises. This is the only term that a price can influence, because it is the only term whose value depends on volume.

```
@dataclass
class CostParams:
    c_var: float
    c_r: float
    C_p: float
    c_0: float
    Phi: float

def floor_price(N: float, job: Job, pl: Place, dp: DeliveryPoint,
                cp: CostParams) -> float:
    ph = p_hat(pl, dp)
    a = addressability(pl, today())
    Am = addressing_mult(a, MU, LAMBDA)
    loss = expected_loss(job, dp, pl)
    numerator = (cp.c_var
                 + cp.Phi * cp.C_p / max(N, 1.0)
                 + (1 - ph) * cp.c_r
                 + loss)
```

```
    return numerator / (ph * Am)
```

9 The Density–Price Fixed Point

9.1 The problem

Price determines volume; volume determines density; density determines the floor; the floor determines the price. A *stable price* is a fixed point:

$$p^* = f\big(N(p^*)\big). \tag{3}$$

9.2 Monotonicity

Lemma 1 (Floor is decreasing in density)**.** *$f(N)$ is strictly decreasing in N.*

Proof. The numerator contains $\Phi C_p/N$ with $\Phi > 0$, $C_p > 0$; all other numerator terms and the denominator are independent of N. The derivative of the numerator with respect to N is $-\Phi C_p/N^2 < 0$, and the denominator is positive. Hence $f'(N) < 0$. □

Lemma 2 (Acceptance is decreasing in price)**.** *For the logistic demand model $N(p) = 1 + (N_{\max} - 1) \cdot \sigma(\alpha + \beta(\mathit{ceiling} - p))$, $N'(p) < 0$.*

Proof. The logistic function is increasing in its argument; the argument decreases in p when $\beta > 0$. Composition preserves direction. □

Theorem 1 (Monotone residual)**.** *Define $h(N) = N - N\big(f(N)\big)$. Then h is strictly increasing in N.*

Proof. f is strictly decreasing (Lemma 1), so $f(N_1) > f(N_2)$ whenever $N_1 < N_2$. $N(\cdot)$ is decreasing (Lemma 2), so $N(f(N_1)) \le N(f(N_2))$. Therefore $h(N_1) - h(N_2) = (N_1 - N_2) - [N(f(N_1)) - N(f(N_2))] < 0$. □

9.3 Existence and solution

Theorem 2 (Fixed-point existence)**.** *If $h(1) \le 0 \le h(N_{\max})$, then there exists $N^* \in [1, N_{\max}]$ with $h(N^*) = 0$, and $p^* = f(N^*)$ is a stable equilibrium.*

Proof. Immediate from the intermediate value theorem and monotonicity (Theorem 1). □

Corollary 1 (Three regimes)**.** *The following three cases exhaust the possibilities:*

1. *$h(1) > 0$: even at unit density, demand is below the floor.* ***Death spiral.*** *No stable interior price exists.*
2. *$h(N_{\max}) < 0$: demand exceeds capacity at any price within the range.* ***Capacity-bound.*** *Price at $f(N_{\max})$ and ration.*
3. *$h(1) \le 0 \le h(N_{\max})$:* ***stable equilibrium.*** *Price at $p^* = f(N^*)$.*

9.4 Solver

Bisection on N is used because h is monotone and the root is unique.

```
def solve_density(job, pl, dp, cp, segment, N_max=40) -> tuple[float, float]:
    def h(N):
        p = floor_price(N, job, pl, dp, cp)
        N_expect = 1 + (N_max - 1) * acceptance_probability(p, job, segment)
        return N - N_expect

    lo, hi = 1.0, float(N_max)
    if h(lo) > 0:
        return (lo, floor_price(lo, job, pl, dp, cp), "SPIRAL")
    if h(hi) < 0:
        return (hi, floor_price(hi, job, pl, dp, cp), "CAPACITY")
    for _ in range(BISECTION_ITERS): # ~40 iters for 1e-9 precision
        mid = 0.5 * (lo + hi)
        if h(mid) < 0: lo = mid
        else: hi = mid
    N_star = 0.5 * (lo + hi)
    return (N_star, floor_price(N_star, job, pl, dp, cp), "STABLE")
```

The `BISECTION_ITERS` constant is set to 40, which yields a solution in N to precision $\approx 2^{-40}$ over the interval $[1, N_{\max}]$. This is more than sufficient for pricing decisions.

10 The Six Gates

Definition 2 (The gates)**.** *For an accepted job at density N with base B and net fee $n = \max(p^*, B)$:*

$$\textit{Gate 1 (address):}\ a \geq a_{\min}(n, T) \tag{4}$$

$$\textit{Gate 2 (floor):}\ B \geq M \tag{5}$$

$$\textit{Gate 3 (contribution):}\ \hat{p}A(a)B > c_{var} + (1-\hat{p})c_r + \Phi C_p/N + \mathbb{E}[loss] \tag{6}$$

$$\textit{Gate 4 (density):}\ N \geq n^* = c_0/(\hat{p}A(a)B - c_{var} - (1-\hat{p})c_r - \Phi C_p/N) \tag{7}$$

$$\textit{Gate 5 (schedulability):}\ \mathrm{Var}[\tau] \leq \Psi(W) \tag{8}$$

$$\textit{Gate 6 (risk):}\ \mathbb{E}[loss] < \textit{contribution} \times \textit{buffer} \tag{9}$$

The gates are tested cheapest-first, so that a job that will fail a late gate is rejected before the expensive computation. Gate 1 requires only the place lookup; Gate 6 requires the full loss model.

10.1 The verdict

```
def prove_job(job, pl, dp, cp, segment, route_density=None) -> QuoteDecision:
    a = addressability(pl, today())
    ph = p_hat(pl, dp)
    Am = addressing_mult(a, MU, LAMBDA)
    loss = expected_loss(job, dp, pl)
    N = route_density or solve_density(job, pl, dp, cp, segment)[0]
    p_star = floor_price(N, job, pl, dp, cp)
    gates = {}

    a_min = a_min_for(job)
    gates["address"] = (a >= a_min, a, a_min)

    b = base_rate(job, pl)
    gates["floor"] = (b >= RATE_CARD[pl.zone]["M"], b, RATE_CARD[pl.zone]["M"])
```

```
    contrib = ph * Am * b - (cp.c_var + (1 - ph) * cp.c_r
                             + cp.Phi * cp.C_p / N)
    gates["contribution"] = (contrib > loss, contrib, loss)

    n_star = cp.c_0 / max(contrib, 1e-6)
    gates["density"] = (N >= n_star, N, n_star)

    tau_m, tau_v = tau_stats(pl, dp.ap_id, hour())
    v_max = var_bound(job.window_width_min)
    gates["schedulability"] = (tau_v <= v_max, tau_v, v_max)

    gates["risk"] = (loss < contrib * RISK_BUFFER, loss,
                     contrib * RISK_BUFFER)

    ceiling = segment["ceiling"]
    if all(g[0] for g in gates.values()):
        return _accept(job, max(p_star, b), p_star, ceiling, N, gates,
                       Verdict.PROVE)
    if p_star <= ceiling:
        return _accept(job, p_star, p_star, ceiling, N, gates, Verdict.PRICE)
    if _preparable(gates):
        return _reject(job, p_star, ceiling, N, gates, Verdict.PREPARE,
                       reason=_prepare_reason(gates))
    return _reject(job, p_star, ceiling, N, gates, Verdict.PASS,
                   reason=_pass_reason(gates))
```

10.2 The four responses

The verdict is one of four, and the choice between them is where discipline lives.

Verdict	Condition	Meaning	Correct action
Prove	All gates pass	Job is provable as presented	Accept at $\max(p^*, B)$
Price	Floor $\leq$ ceiling; some gate fails	Interval exists; the gate failure is overcome by price	Accept at p^*
Prepare	Floor $>$ ceiling, but a state change closes the gap	The job is in the wrong state	Verify address, consolidate, add security, or change client mix
Pass	No state change closes the gap	The job is not provable at any acceptable price	Decline

"Accept anyway" is not among the four. Every job that enters the book without passing its gates is a subsidy from the profitable jobs to the unprofitable ones. That subsidy is what killed the previous generation.

11 Trip Level: Marginal Insertion

A job that passes all six gates as a drop can still fail as a trip member. The trip-level test is:

Definition 3 (Marginal insertion test)**.** *A job j may be inserted into route R if and only if*

$$g_j > \Delta c_{route}(R, j) + (1 - \hat{p}_j)\kappa_j + \Delta_{cascade}(R, j), \tag{10}$$

where g_j is the job's own contribution, Δc_{route} is the extra driving distance it imposes priced at variable cost per km, κ_j is the delay it imposes on downstream stops, and $\Delta_{cascade}$ is the resulting increase in downstream failure probability.

```
def insert_job(route: Route, job: Job, cp: CostParams) -> Optional[Route]:
    best = None
    for pos in range(len(route.stops) + 1):
        candidate = route.with_insertion(pos, job)
        delta_c = candidate.distance_km - route.distance_km
        delta_c_cost = delta_c * cp.c_var_per_km

        ph = p_hat(candidate.place(job), candidate.dp(job))
        cascade = ((1 - ph) * cp.cascade_per_min
                   * candidate.delay_from(pos))

        g_j = contribution(job, candidate.density)
        marginal = g_j - (delta_c_cost + cascade)

        if marginal > 0 and (best is None or marginal > best.marginal):
            best = Insertion(candidate, marginal, pos)
    return best.route if best else None
```

Iteration order. Jobs are sorted by marginal contribution per unit of route deviation, descending, and inserted greedily. This is a $\frac{1}{2}$-approximation to the optimal route under metric distance—good enough in practice and vastly simpler than solving the VRP exactly.

Why the marginal test matters. A job with a contribution of KSh 2,000 that adds 14 km of detour to a tight route fails the marginal test even though it passes every drop-level gate. The job is fine. The job is in the wrong place. The correct response is **Prepare**—consolidate it onto a different route in the same corridor.

12 Corridor Level

A corridor is a portfolio of trips. Its viability test is:

```
def corridor_viable(trips: list[Route], corridor: Corridor) -> tuple[bool, dict
    ]:
    total_contrib = sum(t.contribution for t in trips)
    dpwh = sum(t.drops for t in trips) / sum(t.vehicle_hours for t in trips)
    prev_dpwh = corridor.dpwh_rolling_7d
    return (
        total_contrib > 0 and dpwh >= prev_dpwh * 0.98,
        dict(total_contrib=total_contrib, dpwh=dpwh, prev=prev_dpwh)
    )
```

The second condition is the one that matters operationally. A corridor that is profitable but declining in drops per vehicle-hour is a corridor that is *decaying*. It will be unprofitable in a few weeks, because the density trend is the leading indicator and profit is the lagging one. The corridor check catches that decay while there is still time to act.

13 Long Haul

Long haul has a structurally different pricing problem, because the fee is paid conditional on the whole trip completing. The expected revenue is therefore:

$$\mathbb{E}[\text{revenue}] = \bar{n} \cdot p(a_o) \cdot p(a_d) \cdot A(a), \tag{11}$$

not the average of the two endpoint reliabilities but the **product**. Two endpoints at 90% each give 81% trip success, not 90%. The degradation is multiplicative.

```
def long_haul_price(job, origin, destination, cp, segment) -> QuoteDecision:
    a_o = addressability(origin, today())
    a_d = addressability(destination, today())
```

```
    ph_o = p_hat(origin, job.dp_origin)
    ph_d = p_hat(destination, job.dp_destination)

    joint_p = ph_o * ph_d
    Am = addressing_mult(min(a_o, a_d), MU, LAMBDA)
    loss = expected_loss(job, job.dp_destination, destination)
    backhaul = backhaul_probability(job, origin, destination)

    if backhaul < BACKHAUL_MIN and not one_way_covers_round_trip(job):
        return _reject(job, 0, segment["ceiling"], 2, {},
                       Verdict.PREPARE, reason="no backhaul")

    numerator = (cp.c_var_trip + cp.Phi * cp.C_p
                 + (1 - joint_p) * cp.c_r + loss)
    floor = numerator / (joint_p * Am)
    ceiling = segment["ceiling"]

    if floor <= ceiling:
        return _accept(job, floor, floor, ceiling, 2, {}, Verdict.PROVE)
    if hub_available(origin, destination):
        return _reject(job, floor, ceiling, 2, {}, Verdict.PREPARE,
                       reason="route via hub to raise joint reliability")
    return _reject(job, floor, ceiling, 2, {}, Verdict.PASS,
                   reason="unprovable")
```

The hub check is the operational payoff of the theory. When joint reliability is too low for a direct haul, the engine does not say "refuse"—it says "route via hub," which converts a two-endpoint multiplicative problem into a high-reliability trunk plus a short last-mile job with its own proof. Hub-and-spoke is not a convention in this engine. It is the correct response to multiplicative endpoint reliability when one endpoint is informal and the other is not.

14 Complete Algorithm

Algorithm 1 Quote a job

Input: job j
Retrieve place $pl \leftarrow j$.place_id
Retrieve delivery point $dp \leftarrow j$.dp_id
Retrieve cost params $cp \leftarrow$ cost_params(pl.corridor)
Retrieve segment model $seg \leftarrow$ segment_model(j.segment)
if j is long haul **then**
    $D \leftarrow$ LongHaulPrice(j, pl, j.dest$, cp, seg$)
else
    $D \leftarrow$ ProveJob(j, pl, dp, cp, seg)
end if
Append(quote_log, D) ▷ including all intermediates
return D

15 Acceptance Tests

The engine is correct when the following twelve tests pass on historical data. Each is a unit test that can be run against a preproduction environment.

Test 12 is the one to insist on. The engine must be able to say "no stable price exists" rather than returning a number that happens to be the output of a bisection that never

Algorithm 2 Prove a job

$a \leftarrow \textsc{Addressability}(pl)$
$\hat{p} \leftarrow \textsc{Posterior}(pl, dp)$
$A_m \leftarrow \textsc{AddressingMultiplier}(a)$
$L \leftarrow \textsc{ExpectedLoss}(j, dp, pl)$
$(N, p^*, \text{regime}) \leftarrow \textsc{SolveDensity}(j, pl, dp, cp, seg)$
$B \leftarrow \textsc{BaseRate}(j, pl)$
$n \leftarrow \max(p^*, B)$
$g \leftarrow \hat{p} \cdot A_m \cdot B - (c_{\text{var}} + (1 - \hat{p})c_r + \Phi C_p/N)$
for $k \in$ {address, floor, contribution, density, schedulability, risk} **do**
    if $\textsc{Gate}_k(j, pl, dp, a, \hat{p}, g, n, N) =$ fail **then**
        break
    end if
end for
if all gates passed **then**
    return $(n, \text{PROVE}, \text{all terms})$
else if $p^* \leq$ ceiling **then**
    return $(p^*, \text{PRICE}, \text{all terms})$
else if $\textsc{Preparable}(j, \text{failing gates})$ **then**
    return $(\perp, \text{PREPARE}, \text{prepare reason})$
else
    return $(\perp, \text{PASS}, \text{pass reason})$
end if

converged. A pricing engine that always returns a price is a pricing engine that has never encountered a market it could not enter, which is not the market we operate in.

16 Calibration and Cold Start

Three of the engine's inputs cannot be known on day one.

Shadow mode. Run the engine in shadow for the first month. It quotes alongside the human. Where they disagree, the parameter that produced the disagreement is the one to investigate. This is the operational form of the sanity baseline, and it is why the audit log matters from the first day: it allows historical decisions to be re-run with corrected parameters, and the resulting counterfactual is the evidence base for each parameter change.

17 Complexity and Performance

The online path is dominated by SolveDensity, which is 40 bisection iterations of arithmetic on eight floats. At 40 iterations and 50ns per iteration, the online path is $O(2\,\mu\text{s})$ plus I/O, comfortably inside the 50ms budget.

18 Limitations

Three honest limitations.

The ceiling is not derivable. Willingness to pay is empirical. The engine gives the floor and the allocation; it cannot tell you what a distributor will pay for reliability. This is why the engine is a feasibility test plus an allocation rule, not a price oracle.

The fixed point may not exist. In a corridor where density cannot be built—dispersed one-off consumer drops, long haul without backhaul—no stable price exists, and the correct

#	Test	Expected
1	Mass-market drop, $a = 0.25$, $\bar{n} = 800$	PASS (below $a^* \approx 0.36$)
2	Same drop, $a = 0.775$ (verified place)	PROVE
3	B2B contract, $\bar{n} = 2{,}000$, $a = 0.2$	PROVE (above $a^* \approx 0.17$)
4	$\bar{n} = 500$, $M = 600$	PRICE (raise to floor)
5	Profitable job, 14 km detour, tight route	PASS at trip level
6	Same job, consolidation window available	PREPARE
7	Corridor with falling DPWH	`corridor_viable=False`
8	Long haul, $a_d = 0.2$, no backhaul	PREPARE (route via hub)
9	Long haul, $a_d = 0.9$, backhaul confirmed	PROVE
10	Price cut -10% in dense corridor	DPWH rises $\rightarrow$ self-financing
11	Price cut -10% in long haul	DPWH flat $\rightarrow$ spiral flagged
12	Fixed point with $h(1) > 0$	Returns `SPIRAL`, not a price

Input	Cold-start fallback	When real data arrives
Segment ceilings	Interview-derived, wide range	Logistic fit on accept/decline history
τ variance per place	Global default	After $n \geq 3$ drops at that place
Φ, c_{var}, c_0	Management estimate	After four weeks of trip P&L

output is not a number but a verdict. The engine must be allowed to return `SPIRAL` or `CAPACITY`. If management overrides that verdict, the override must be logged with a reason and a review date.

Competition is exogenous. The engine prices against cost and customer value, not against a rival's response. In a corridor with a subsidised competitor, the floor may be above the market price and the engine will correctly say "pass"—which may be the wrong strategic answer if the competitor is irrational and will fail. The engine says so, and the override is a management decision, not an algorithmic one.

19 Conclusion

The Peleka Universal Rate Model is a decision procedure, not a price. It answers one of four questions for every job: prove, price, prepare, or pass. Its defining feature is that it can answer *no stable price exists*—a verdict the previous generation of logistics companies could not produce, because they did not believe in it, and that belief is what killed them.

The engine is universal in structure and local in parameters. The same object prices a kiosk drop and a trunk haul, a Nairobi delivery and a Mombasa haul, a standard parcel and a high-value device. What changes are the inputs, not the algorithm. The rate card is what the customer sees. The floor is what the business needs. The gap between them is where the work of *prepare* lives.

Every decision is logged with every intermediate term. The log is the audit trail, and the audit trail is the asset. Firms that keep it can re-run history and discover what they would have done. Firms that do not keep it will be surprised by their own past.

> **The algorithm in one sentence:** score the job from the ledger; solve for the density at which the floor that density implies produces the acceptance that density assumes; if no such density exists, say so; otherwise test six gates and return one

Operation	Complexity	Notes
Addressability	$O(1)$	Reads stored fields
Posterior	$O(1)$	Beta arithmetic
SolveDensity	$O(\log(1/\epsilon))$	Bisection; 40 iterations sufficient
ProveJob	$O(\log(1/\epsilon))$	Dominated by density solve
InsertJob	$O(N \cdot \lvert R \rvert)$	N jobs, $\lvert R \rvert$ stops per route
CorridorViable	$O(\lvert\text{trips}\rvert)$	Linear scan

of four verdicts; and log every term so the engine can be audited against its own history.